

VULTR HACKATHON SUBMISSION

Run it. Prove it. Retire the box.

Defensive repository verification with observed exploits, isolated execution, and a disposable second chance to prove a fix.

Real execution

Canary evidence

Disposable VX1



THE GAP

A model can suspect a flaw.
Only execution can prove one.

SOURCE-ONLY CLAIM

"This route looks vulnerable."

A plausible code path is useful for triage, but it is not evidence that an exploit worked in the running app.

ENVIRONMENT-OWNED PROOF

"This request returned our canary."

Run the app in isolation, send a real test request, and let observed behavior decide whether to report a finding.

Our rule: no finding without a proof.

ONE RUN, BOUNDED FROM START TO FINISH

Find → prove → patch → re-prove.

01 / INTAKE

Choose a target

Allowlisted seeded app or a repo path / HTTPS clone address.

04 / CONFIRM

Fire real HTTP tests

Report a finding only when a planted, non-sensitive canary is observed.

02 / ISOLATE

Start a disposable VX1

Fetch staged source, run the target with gVisor or Microsandbox, and health-check over the VPC.

05 / REMEDIATE

Patch a copy

Optional re-exploitation runs on a second disposable VX1 in the same runtime.

03 / TRIAGE

Rank candidate sinks

Recon and static analysis feed Vultr Inference; an honest offline fallback exists.

06 / CLEAN UP

Verify deletion

Check the VX1 returns 404; delete staged source and return bounded results.

THREE ROLES, ONE EXECUTION BOUNDARY

The target never needs the control plane's cloud keys.

PUBLIC-FACING DEMO TIER

Web page + proxy

Password-gated browser UI;
server-side proxy holds the control
bearer.



PERSISTENT VX1 · CHICAGO

Control plane

FastAPI jobs, Vultr API, inference, repo
source, and per-run readiness registry.



DISPOSABLE VX1 · PER SCAN

Sandbox host

Runs target on a verified VPC address,
then destroys the host after the job.

Web → control: NetBird private operator link · Control ↔ sandbox: Vultr VPC · Source transfer: scoped, presigned Object Storage GET

INSIDE THE DISPOSABLE VULTR VX1

Choose the boundary for the workload.

OPTION 1 / CONTAINER

gVisor · runc

A userspace syscall boundary around the running app. Images are built under runc on the disposable host, then run with `--runtime=runc`.

OPTION 2 / KVM MICROVM

Microsandbox

A separate guest-kernel boundary. The target and its dependency installation run inside the microVM on the same disposable VX1.

Both modes use a disposable outer VX1. Neither runtime by itself proves blanket egress isolation; the current microVM path keeps its default network posture.

THE ENVIRONMENT IS THE JUDGE

The model ranks. The canary decides.

- 01 Recon and static sweep identify reachable candidate sinks.
- 02 Vultr Inference ranks a test plan; offline heuristic is labeled when used.
- 03 A real confirmer sends HTTP probes to the running app.
- 04 Only an observed, request-attributed canary becomes a finding.

```
candidate → SQL sink
request   → test payload fired
response  → canary observed
finding   → confirmed
```

```
no canary ≠ app is safe
```

REAL-STACK RUNS RECORDED IN README · 2026-09-27

What the prototype has actually shown.

5/5

Confirmed classes on each of
two seeded apps

gVisor scans with live Vultr Inference

5/5

Certified closed in a
documented remediation run

Re-exploit on a second disposable VX1

+3

Host drop-counter change
with matching kernel log

Controlled VPC sandbox smoke probe

404

Independent check after VX1
destruction

Cleanup confirmed for recorded runs

Separate demonstrations: controlled network smoke is not a general egress proof; seeded app success is not arbitrary-repo coverage. Microsandbox also returned 5/5 seeded findings in a recorded run.

A FIX HAS TO SURVIVE THE SAME TEST

Patch a disposable copy. Re-run the exploit.

PASS ONE · FIND

Prove the breach

Run the original target on VX1 #1, capture a canary-confirmed exploit, and destroy that VX1.



PATCH · CONTROL PLANE

Prepare the fix

Apply accepted candidate patches to one cumulative disposable source copy; leave the original untouched.



PASS TWO · VERIFY

Re-exploit on VX1 #2

Block the prior exploit, preserve functional behavior, validate the patch, then destroy the second VX1.

“Certified closed” means those tested exploit paths were re-checked—not a guarantee that the repository is safe from all vulnerabilities.

SHOW THE CORRECT SURFACE FOR EACH CLAIM

Three interfaces. Different evidence.

LIVE-PAGE CODE PATH

Sandbox job UI

`site/live.html` calls a password-gated proxy, which arms and starts a real `sandbox_scan` through the control API.

INTERACTIVE LOCAL DEMO

Animated console

The finder and patch loop are real, but this console starts its target as a local subprocess. Its VM animation is not live sandbox execution.

PRESENTATION ONLY

Showcase site

`site/index.html` tells the story using captured results; it is not connected to the backend.

THE ORCHESTRATION CENTER, NOT JUST A PLACE TO HOST A PAGE

Vultr powers every critical step.

01 / CLOUD COMPUTE

Persistent control + disposable VX1s

One private planner; fresh host for target execution and another when re-exploitation is requested.

02 / SERVERLESS INFERENCE

Reachability-aware triage

The model ranks candidate sinks into a test plan; recorded results disclose the model path.

03 / VPC

Private control-to-sandbox path

Provider-verified guest VPC IP, callback-only listener, and a private target endpoint.

04 / OBJECT STORAGE

Scoped source transfer

Deterministic source tarball staged privately and fetched with a short-lived single-object presigned GET.

WHAT WE WILL NOT CLAIM

Strong evidence has a defined scope.

Arbitrary repos are not 5/5 by default

SQLi and IDOR require app-specific canaries. Manifest-less environmental canaries currently need to move from the control host into the remote sandbox.

Network containment is not universal

Controlled drop evidence is real; it does not prove all egress is denied. Microsandbox retains its default network posture.

Some trust boundaries remain to harden

gVisor image builds execute under runc on the disposable host. The web demo's password can authorize sequential billable jobs.

Prototype records are not signed receipts

Job state is in memory; durable history, signed commit-bound receipts, and production-grade auth and spending controls remain future work.

CERBERUS

Proof, not promises.

The model proposes where to look. The running target and canary provide the verdict. The disposable host is destroyed.

See a finding

Inspect its actual request and redacted canary evidence.

See the fix

Re-exploit the patched disposable copy on a second VX1.

See the cleanup

Verify the VX1 returns 404 and staged source is removed.